CHAPTER 1

Introduction to DBMS

Database · DBMS · file system vs DBMS · three-schema architecture · data independence · languages · components

QUICK TRICK

Quick tricks are in the green boxes. This book teaches the DBMS part of GATE CSE from zero. Learn the simple idea first, then the exam point.

The plan for these DBMS notes

Chapter	Topic
1	Introduction to DBMS (this chapter)
2	ER model
3	Relational model and keys
4	Relational algebra and calculus
5	SQL
6	Functional dependencies and normalization
7	Transactions and concurrency control
8	Recovery
9	File organization, indexing, B and B+ trees
10	GATE quick revision
11	Practice questions with answers

1.1 Data, database and DBMS

● **LEVEL 1 · BEGINNER**

Data means raw facts, like a name, a number or a date. Example: *Ravi, 21, 87*. By itself it does not say much.

Information is data that has a meaning. Example: *Ravi is 21 years old and scored 87 marks*.

A **database** is an **organized collection of related data**. Example: all the students, courses and marks of a college.

A **DBMS (Database Management System)** is the **software** that creates the database and lets us store, find, change and protect the data. Examples: MySQL, Oracle, PostgreSQL, SQL Server.

Analogy. A library. The **books** are the data (the database). The **librarian and the rules** are the DBMS. You do not walk into the store room and search the shelves yourself. You ask the librarian.

Term	Simple meaning	Example
Data	Raw facts	Ravi, 21, 87
Database	Organized collection of related data	College records

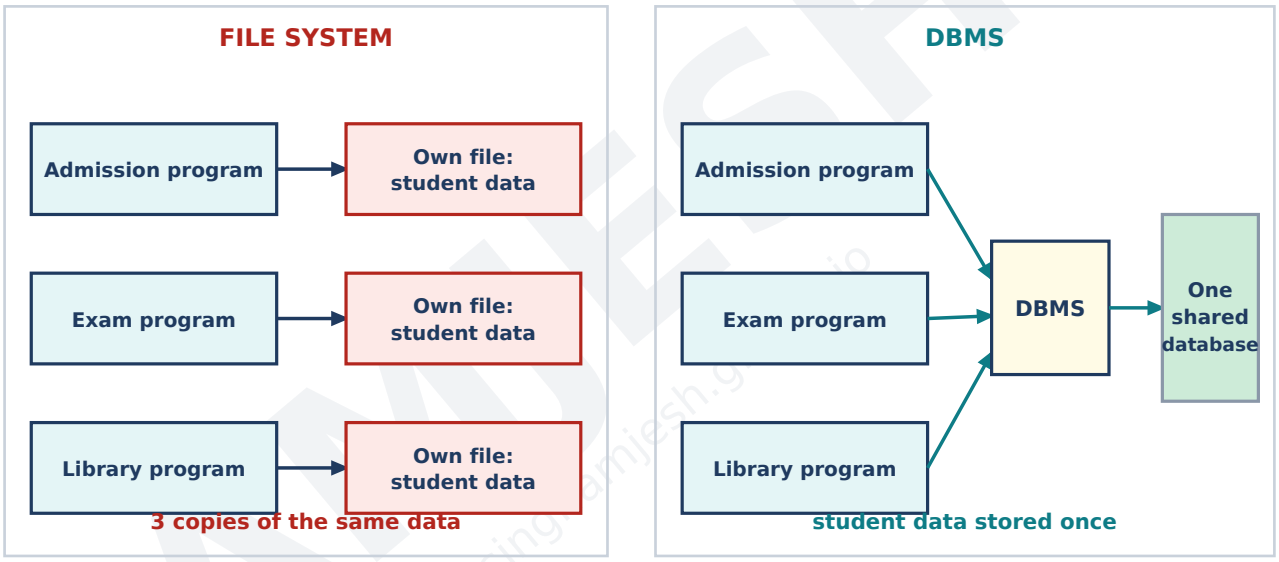
Term	Simple meaning	Example
DBMS	Software that manages the database	MySQL, Oracle
Database system	Database + DBMS + application programs	The whole college system
Metadata	Data about data (names of tables, columns, types)	Student(RollNo INT, Name TEXT)

EASY MEMORY
Database = the data. DBMS = the software that manages the data. Metadata = data about the data.

1.2 File system vs DBMS

● LEVEL 1 · BEGINNER

Before DBMS, programs stored data in plain **files**. Each program had its own files. This caused many problems:



Problem of the file system	What goes wrong	Simple example
Data redundancy and **inconsistency**	The same data is stored many times. Copies stop matching.	A student's address changed in one file but not in the others
Difficulty in accessing data	Every new question needs a new program.	“List students with marks above 80” needs a new program
Data isolation	Data is in different files and formats.	Hard to combine student data and library data
Integrity problems	Rules (like “marks ≤ 100”) are hidden inside programs.	A new program forgets the rule
Atomicity problems	A task stops in the middle, leaving half-done work.	Money taken from one account but not added to the other
Concurrent access anomalies	Two users change the same data at the same time.	Two clerks update the same balance together
Security problems	Hard to give each user only the data he may see.	A clerk can open the whole salary file

QUICK TRICK

Remember the 7 problems in 3 groups. (1) **Too many copies:** redundancy, inconsistency, isolation. (2) **Not safe:** integrity, atomicity, concurrent access, security. (3) **Hard to use:** difficult access to data.

Advantages and disadvantages of a DBMS

Advantages	Disadvantages
Less redundancy, more consistency	Cost: software, hardware and trained people
Data sharing between many users	Complex to design and manage
Integrity rules enforced in one place	Slower for very simple jobs (extra layers)
Security and access control	A failure can stop everything (single point of failure)
Backup and recovery	Needs regular backups and tuning
Data independence (see 1.4)	

1.3 Data abstraction: three levels

● **LEVEL 2 · EXAM**

Idea. A user should not need to know **how** the data is stored. A DBMS hides the details in **levels**. The user sees only what he needs.

Level	Also called	What it describes	Who sees it
Physical	Internal level	**How** data is stored: files, records, indexes, disk blocks	Database system programmers
Logical	Conceptual level	**What** data is stored and the relationships: tables, columns, keys, constraints	DBA
View	External level	Only the **part** of the database a user needs	End users and application programs

QUICK TRICK

Order from the bottom: Physical → Logical → View. Think of a car: the **engine** is the physical level (how it works), the **whole car design** is the logical level (what parts exist), the **dashboard** is the view (what the driver sees).

Schema and instance

	Schema	Instance
Meaning	The **design** (structure) of the database	The **data** stored at one moment
Changes	Rarely	Every time we insert, update or delete
Analogy	The blueprint of a building	The building with people inside at 10 a.m.
Programming analogy	Variable declaration (int x)	The value now (x = 5)
Example	Student(RollNo, Name, Marks)	The rows in Student right now

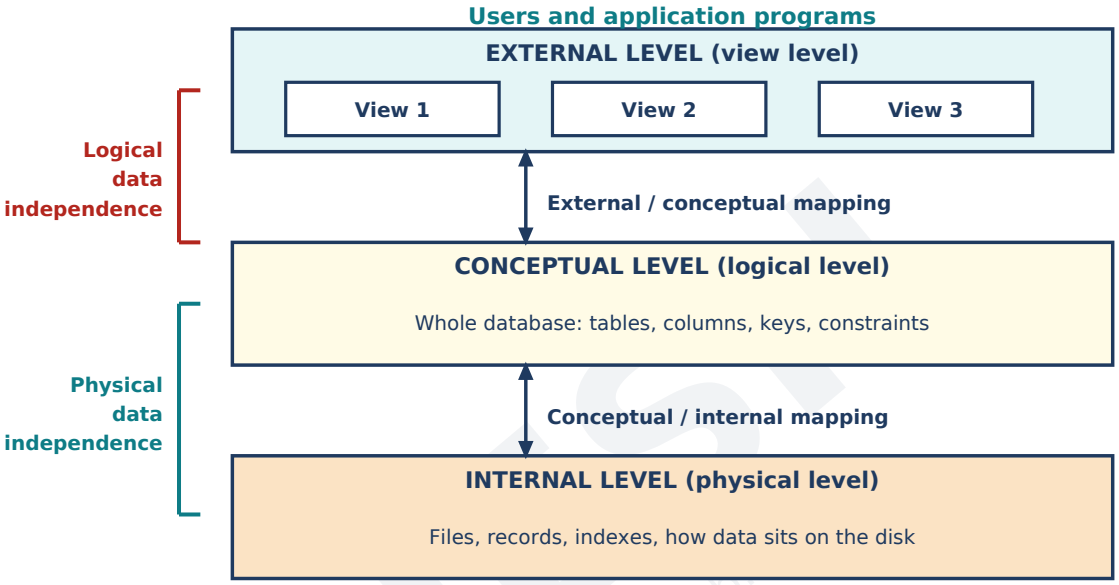
There is one schema at **each** level: physical (internal) schema, logical (conceptual) schema, and view (external) schemas. A database has **one** internal schema, **one** conceptual schema and

many external schemas (one per kind of user).

EASY MEMORY
Schema = structure (blueprint). Instance = content (the data right now). The schema is not the same as the instance. A table definition is the schema; the rows are the instance.

1.4 Three-schema architecture and data independence

● LEVEL 2 · EXAM



The levels are connected by **mappings**: the **external/conceptual** mapping and the **conceptual/internal** mapping. When a lower level changes, only the mapping is updated, and the upper levels do not notice.

	Physical data independence	Logical data independence
Meaning	Change the **physical** (internal) schema **without changing** the logical schema	Change the **logical** (conceptual) schema **without changing** the views and application programs
Example	Add an index, change the file structure, move data to a new disk	Add a new column to a table, split a table in two
Difficulty	Easier	**Harder**
Between levels	Conceptual and internal	External and conceptual

QUICK TRICK
“Change below, nothing above notices.” Physical independence: the logical level does not notice a change in storage. Logical independence: the views do not notice a change in the logical design. **Logical independence is harder**, because programs depend heavily on the logical structure.

GATE POINT
Counting: three levels, **two mappings**. A database has 1 internal schema, 1 conceptual schema and many external schemas.

1.5 Database languages

● LEVEL 1 · BEGINNER

A DBMS gives us different **sub-languages** for different jobs. In practice they are all part of **SQL**.

Language	Job	Commands	Easy memory
DDL (Data Definition Language)	**Define** or change the structure (schema)	CREATE, ALTER, DROP, TRUNCATE, RENAME	**C-A-D-T-R**
DML (Data Manipulation Language)	**Use** the data: read and change	SELECT, INSERT, UPDATE, DELETE	**S-I-U-D**
DCL (Data Control Language)	**Control** who can do what (permissions)	GRANT, REVOKE	**G-R**
TCL (Transaction Control Language)	**Control transactions**	COMMIT, ROLLBACK, SAVEPOINT	**C-R-S**

QUICK TRICK

One-line test: Structure → DDL. **Data** → DML. **Permission** → DCL. **Save/undo** → TCL.

GATE POINT

GATE point: TRUNCATE is DDL (it removes all rows at once and cannot normally be rolled back), while DELETE is DML (can be rolled back and can have a WHERE condition). SELECT is usually counted in DML (some books call it DQL, Data Query Language).

Procedural and declarative (non-procedural) languages

	Procedural	Declarative (non-procedural)
The user says...	**How** to get the result, step by step	**What** result he wants
Example	Relational algebra, C or Java programs	SQL, relational calculus
Who finds the method?	The user	The DBMS (query optimizer)

EASY MEMORY

SQL says WHAT, not HOW.

1.6 Data models

● LEVEL 1 · BEGINNER

A **data model** is a set of ideas for describing data, its relationships and its rules. It tells us **how the database is organized**.

Model	How data is organized	Remark
Hierarchical	A tree (one parent, many children)	Old, fast for tree-like data
Network	A graph (a record can have many parents)	Old, complex pointers
Relational	**Tables** (relations) of rows and columns	**The most used**. Most of GATE is based on it.
Entity–Relationship (ER)	Entities and relationships (a design picture)	Used to **design** the database (Chapter 2)

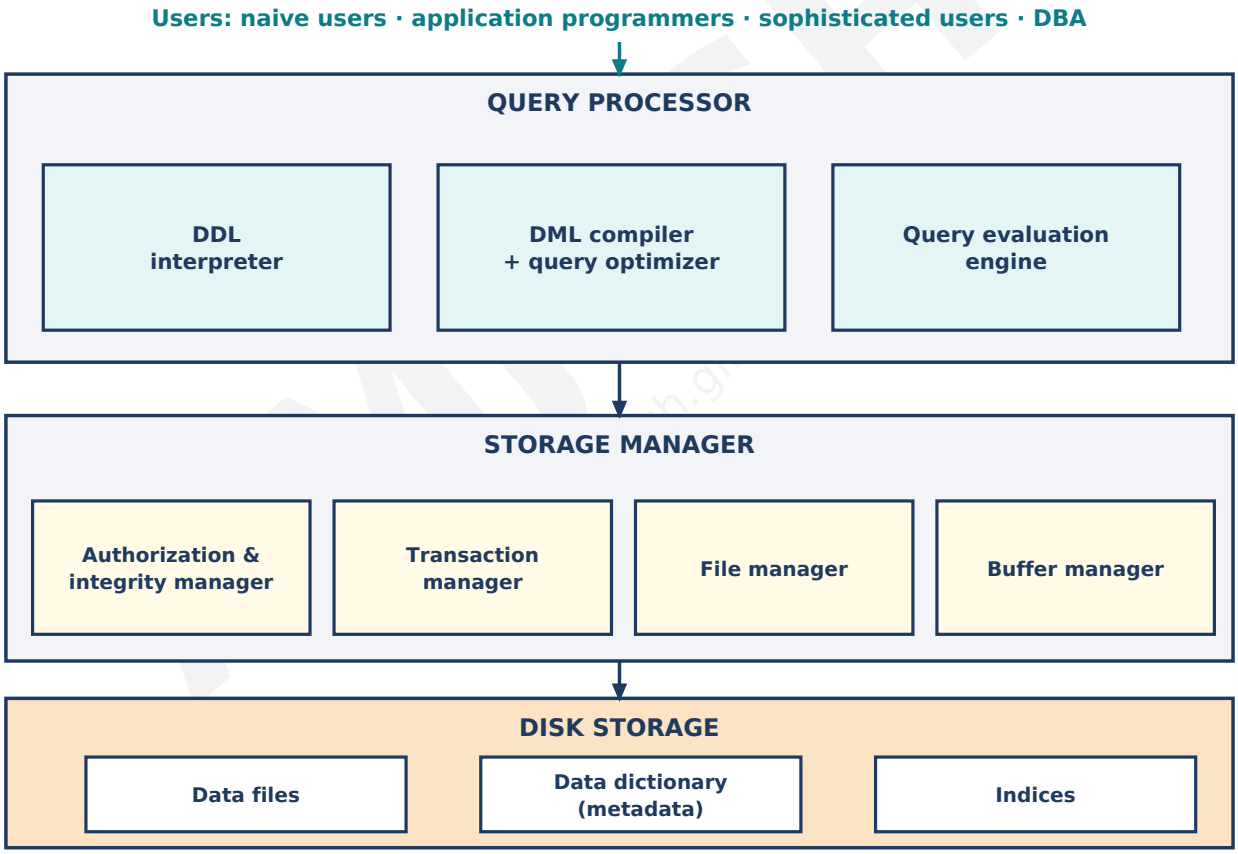
Model	How data is organized	Remark
Object-oriented / object-relational	Objects with attributes and methods	Used for complex data
Semi-structured (XML, JSON, document)	Flexible, self-describing documents	NoSQL systems such as MongoDB

QUICK TRICK
For GATE: remember **relational = tables**, **ER = design picture**, **hierarchical = tree**, **network = graph**.

1.7 DBMS components and architecture

● **LEVEL 2 · EXAM**

Inside a DBMS there are two main parts. The **query processor** understands and plans the user's request. The **storage manager** actually reads and writes the data on the disk and protects it.



Component	What it does
DDL interpreter	Reads DDL statements and records the schema in the data dictionary
DML compiler + query optimizer	Turns a query into a low-level plan and chooses the **cheapest** plan
Query evaluation engine	Runs the chosen plan
Authorization and integrity manager	Checks permissions and integrity rules
Transaction manager	Makes sure transactions are correct: concurrency control and recovery

Component	What it does
File manager	Manages the space on the disk
Buffer manager	Keeps frequently used disk blocks in main memory
Data dictionary (catalog)	Stores **metadata**: table names, columns, types, constraints, users
Indices	Extra structures for fast searching (Chapter 9)

QUICK TRICK
Query processor = understands the question. Storage manager = fetches and protects the data. The **optimizer** is part of the query processor. The **buffer manager** and **transaction manager** are part of the storage manager.

Users of a database

User	What he does
Naive (end) users	Use ready-made applications (ATM, ticket booking)
Application programmers	Write programs that use the database
Sophisticated users	Write queries directly (for example analysts using SQL)
Database administrator (DBA)	Controls the whole database

Duties of the DBA: design the logical and physical schema, decide the storage structure and indexes, give permissions (security), take backups and recover from failures, and keep the system fast (tuning).

Application architecture: 2-tier and 3-tier

2-tier



3-tier



	2-tier	3-tier
Layers	Client and database server	Client, **application server**, database server
Client talks to	The database directly	The application server only
Business logic	In the client	In the **application server**
Used for	Small systems	Web applications (more secure, easier to scale)

Quick-Solve Tricks — Summary

Topic	Trick
Database / DBMS / metadata	Data / software that manages the data / data about data.
File system problems	3 groups: too many copies, not safe, hard to use.
Three levels	Physical (how) → Logical (what) → View (part).
Schema vs instance	Schema = blueprint. Instance = data now.
Data independence	Change below, nothing above notices. Logical is harder.
Languages	Structure = DDL (C-A-D-T-R). Data = DML (S-I-U-D). Permission = DCL. Save/undo = TCL.
TRUNCATE	DDL, not DML.
SQL	Declarative: says WHAT, not HOW.
Components	Query processor (understand) + storage manager (fetch and protect).
Tiers	3-tier has an application server in the middle.

GATE QUICK CHECK — Chapter 1

```
Levels : Physical (internal) < Logical (conceptual) < View (external)
Schemas: 1 internal, 1 conceptual, many external      Mappings: 2
DDL : CREATE ALTER DROP TRUNCATE RENAME     DML : SELECT INSERT UPDATE DELETE
DCL : GRANT REVOKE                          TCL : COMMIT ROLLBACK SAVEPOINT
```

Traps: schema is not the same as instance · TRUNCATE is DDL · logical independence is harder than physical · data dictionary stores metadata, not the user's data.

Practice questions

● **LEVEL 1 · BEGINNER**

Q1. Which of these is a DDL command: CREATE, SELECT, GRANT, COMMIT?

Q2. What is metadata? Where is it stored?

Q3. How many levels does the three-schema architecture have? Name them.

● **LEVEL 2 · EXAM**

Q4. Classify each command as DDL, DML, DCL or TCL: ALTER, UPDATE, REVOKE, ROLLBACK.

Q5. A student's address is stored in two files. It is changed in only one file. Which file-system problem is this?

Q6. Is each of these a schema or an instance? (a) Student(RollNo, Name, Marks). (b) The 500 rows of the Student table at 10 a.m.

● **LEVEL 3 · GATE**

Q7. Which data independence is harder to achieve: logical or physical?

Q8. A DBA adds a new index to speed up queries, and no application program needs to be changed. Which kind of data independence is this?

Q9. Two clerks withdraw money from the same account at the same time. Which part of the DBMS keeps the data correct?

Q10. In a 3-tier architecture, where does the business logic normally run?

Q11. Complete: SQL is a declarative language, so the user tells the DBMS ___ he wants and not ___ to get it.

Q12. Which statement is false? (a) A DBMS can reduce redundancy. (b) A DBMS needs no backup because it never fails. (c) A DBMS provides access control. (d) A DBMS helps data sharing.

Answers

Q	Solution
1	**CREATE** (it defines structure). SELECT is DML, GRANT is DCL, COMMIT is TCL.
2	Metadata is **data about data** (table names, column types, constraints). It is stored in the **data dictionary (catalog)**.
3	**Three**: external (view), conceptual (logical) and internal (physical) level.
4	ALTER = **DDL**. UPDATE = **DML**. REVOKE = **DCL**. ROLLBACK = **TCL**.
5	**Data inconsistency**, caused by data redundancy (the same data stored twice).
6	(a) **Schema** (the structure). (b) **Instance** (the data at one moment).
7	**Logical** data independence is harder (programs depend strongly on the logical structure).
8	**Physical** data independence (the internal level changed, the logical level and programs did not).
9	The **transaction manager** (concurrency control) in the storage manager.
10	In the **application server** (the middle tier).
11	**what** he wants and not **how** to get it.
12	**(b)** is false. A DBMS still needs backup and recovery, because failures can happen.

Next: Chapter 2 — ER Model (entities, attributes, relationships, cardinality, participation, weak entities, ER to relational mapping).